Solidity Practice Codes (Remix IDE)

Each section = one file. Under each code you will find a sample input and the output you will see in Remix.

---

1. ArrayExample.sol — Fixed-size array with index access

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract arrayContract{
    uint[] public arr=[3,5,6,7];
    function setArray(uint index, uint value) public{
        arr[index]=value;
    }
    function getArray(uint index) view public returns (uint){
        return arr[index];
    }

    function arrLength() public view returns(uint){
        return arr.length;
    }
}
```

Example (initial array = [3, 5, 6, 7])

Input	Output
`arrLength()`	`0: uint256: 4`
`getArray(2)`	`0: uint256: 6`
`arr(0)`	`0: uint256: 3`
`setArray(1, 10)` **then** `getArray(1)`	`0: uint256: 10` **(array becomes [3, 10, 6, 7])**
`setArray(4, 9)`	**Error: reverts (index out of bounds, Panic 0x32)**

2. calculator.sol — Basic calculator

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Calculator {
    function add(uint256 a, uint256 b) public pure returns (uint256) {
        return a + b;
    }

    function subtract(uint256 a, uint256 b) public pure returns (uint256) {
        require(a >= b, "Result cannot be negative");
        return a - b;
    }

    function multiply(uint256 a, uint256 b) public pure returns (uint256) {
        return a * b;
    }

    function divide(uint256 a, uint256 b) public pure returns (uint256) {
        require(b != 0, "Cannot divide by zero");
        return a / b;
    }
}
```

Example

Input	Output
`add(10, 5)`	`0: uint256: 15`
`subtract(10, 5)`	`0: uint256: 5`
`subtract(5, 10)`	**Revert: "Result cannot be negative"**
`multiply(10, 5)`	`0: uint256: 50`
`divide(10, 5)`	`0: uint256: 2`
`divide(10, 3)`	`0: uint256: 3` **(integer division, no decimals)**
`divide(10, 0)`	**Revert: "Cannot divide by zero"**

3. DynamicArray.sol — Push, pop, length of a dynamic array

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract arrayContract{
    uint[] public arr;
    function pushArrayElement(uint value) public {
        arr.push(value);
    }
    function arrLength() public view returns(uint){
        return arr.length;
    }
    function popArrayElement() public {
        arr.pop();
    }
    function getFullArray() view public returns (uint[] memory){
        return(arr);
    }
}
```

Example (array starts empty)

Input	Output
`pushArrayElement(10)`	**arr = [10]**
`pushArrayElement(20)`	**arr = [10, 20]**
`pushArrayElement(30)`	**arr = [10, 20, 30]**
`arrLength()`	`0: uint256: 3`
`getFullArray()`	`0: uint256[]: 10,20,30`
`arr(1)`	`0: uint256: 20`
`popArrayElement()` **then** `getFullArray()`	`0: uint256[]: 10,20`
`popArrayElement()` **on an empty array**	**Error: reverts (Panic 0x31)**

---

4. loop.sol — For loop filling an array

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract ForContract{
    uint[] public arr;
    function loopInitialization(uint len) public{
        for(uint count=0; count < len; count++){
            arr.push(count);
        }
    }
    function getFullArray() view public returns (uint[] memory){
        return(arr);
    }
}
```

Example

Input	Output
`loopInitialization(5)` then `getFullArray()`	`0: uint256[]: 0,1,2,3,4`
`arr(3)`	`0: uint256: 3`
`loopInitialization(3)` again, then `getFullArray()`	`0: uint256[]: 0,1,2,3,4,0,1,2` (values are appended, not replaced)

---

5. oddEven.sol — Check whether a number is odd or even

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract OddEven{
    string status;
    function odd(uint value) public returns(string memory){
        if(value%2 == 0){
            status="Even";
```

```
        }
        else{
            status="Odd";
        }
        return status;
    }
}
```

Example

Input	Output
`odd(4)`	`Even`
`odd(7)`	`Odd`
`odd(0)`	`Even`

Note: this function is not `view`, so it runs as a transaction. In Remix, click the transaction in the terminal and look at decoded output to see `"0": "string: Even"`.

---

6. struct.sol — Single struct variable

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

struct Student{
    uint roll;
    string name;
}

contract structContract{
    Student public s1;
    constructor(uint _roll, string memory _name){
        s1.roll = _roll;
        s1.name = _name;
    }
    function setter(uint _roll, string memory _name) public {
        Student memory s2 = Student(_roll, _name);
        s1=s2;
    }
```

```
    function getter() public view returns (Student memory){
        return s1;
    }
}
```

Example

Input	Output
Deploy with `1, "Ravi"`	**s1 = (1, "Ravi")**
`s1()`	`roll: 1 , name: Ravi`
`getter()`	`0: tuple(uint256,string): 1,Ravi`
`setter(2, "Amit")` **then** `getter()`	`0: tuple(uint256,string): 2,Amit` **(old value is overwritten)**

---

7. structArray.sol — Array of structs

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

struct student{
    uint roll;
    string name;
}

contract arrayContract{
    student[] public s1;
    constructor(uint _roll, string memory _name){
        student memory s = student(_roll, _name);
        s1.push(s);
    }

    function setter(uint _roll, string memory _name) public {
        student memory s2 = student(_roll, _name);
        s1.push(s2);
    }

    function getter() public view returns(student[] memory){
        return s1;
```

```
    }
}
```

(In your screenshot, lines 1–2 were cut off, so the SPDX line is added here as in your other files.)

Example

Input	Output
Deploy with `1, "Ravi"`	**s1 = [(1, "Ravi")]**
`s1(0)`	`roll: 1 , name: Ravi`
`setter(2, "Amit")`	**s1 = [(1, "Ravi"), (2, "Amit")]**
`getter()`	`0: tuple(uint256,string)[]: [1,Ravi],[2,Amit]`
`s1(1)`	`roll: 2 , name: Amit`

Difference from struct.sol: there `s1=s2` **replaces the single student; here** `s1.push(s2)` **adds a new student to the list.**